Tic Tac Toe AI: Project Brief

The Short Version

You're building a Tic Tac Toe web app with accounts, AI opponents, and a stats page. You'll work on the same project as the rest of the class, but you'll build and ship your own version.

The app is not the point. Everyone's app will work. What matters is how you got there: your checkpoints, your process, your decisions, and the custom features you add at the end.

What You're Learning

1. **Spec-driven development.** Read the requirements, then develop the specs with the help of AI. Follow it while developing. Write one of your own for your custom features.
2. **AI-assisted coding with discipline.** You'll use AI to help you code, but you have to prove you understand what you built.
3. **AI as a product feature.** Your app will call the Groq API to make moves and talk trash (or encouragement) to the player.
4. **GitHub fundamentals.** Commits, tags, `.gitignore`, keeping secrets out of your repo.

The Timeline

Two weeks. 10 school days. Each day follows the same rhythm:

- **Teacher-led day:** We design and demo together as a class.
- **Student work day:** You build your version on your own.

You ship a few checkpoints every couple of days.

What You're Building

A web app where users can:

- Sign up, log in, and log out
- Play Tic Tac Toe against another human (same screen)
- Play Tic Tac Toe against an AI with three difficulty levels and three personalities
- See a leaderboard and AI performance stats
- View a Checkpoints page showing your build process (this is the important one)

Required Tech Stack

- **Node.js** (version 20 or higher)
- **Express** for the server
- **Vanilla HTML, CSS, and JavaScript** for the frontend. No frameworks, no build tools.
- **JSON files** for all data storage (no database)
- **Groq SDK** for AI calls
- **dotenv** for loading `.env`
- **express-session** for login sessions
- **nodemon** for development

No React. No TypeScript. No Tailwind. No databases. If you want to try one of those, build it in your next project.

Required File Structure (Minimum)

tic-tac-toe-ai/

```
├── .env                # your Groq API key (never commit)
├── .env.example        # template showing which keys are needed
├── .gitignore
├── README.md
├── package.json
├── server.js           # main Express server
├── /data               # ALL JSON data lives here, no exceptions
│   ├── users.json
│   └── games.json
└── /public             # everything the browser sees
    ├── index.html
    ├── styles.css
```

└── main.js

Rules (Non-Negotiable)

1. All JSON files and all app data live in `/data`. Nothing outside this folder.
2. All browser-facing files live in `/public`.
3. `server.js` is always the entry point. `npm start` runs it.
4. `.env` is never committed. Ever.

You Can Add More

As your project grows, add files and folders if it helps you stay organized. Examples that are welcome:

- A `/routes` folder once `server.js` gets long
- More HTML pages in `/public` (like `game.html`, `stats.html`, `checkpoints.html`)
- A `/lib` folder for helper functions
- More JSON files inside `/data`

What you cannot do: rename the required files, move JSON outside `/data`, move browser files outside `/public`, or add a framework.

Required `.gitignore` (Minimum)

node_modules/

.env

data/users.json

data/games.json

Security Rules

- Passwords are stored in plaintext in `users.json` **for learning purposes only.** We'll talk in class about why production apps never do this and what they use instead.
- Your Groq API key lives in `.env` and stays out of GitHub. If you commit a key, you have to rotate it immediately.
- Before every commit, run `git status` and look at what you're about to push.

Checkpoints

Each checkpoint is a tagged commit on your `main` branch. Tag format: `CP##-name`.

Tag	Name	What It Proves
CP01-world	Basic App Setup	Server runs, repo has README and `.gitignore`, hello world route works.
CP02-accounts	Register and Login	Users can sign up, log in, log out. `users.json` writes correctly.
CP03-game	Game Board UI	3x3 grid renders, cells respond to clicks, turn indicator works.
CP04-pvp	Human vs Human	Two players can finish a full game. Win and draw detection correct.
CP05-save	Save Games	Finished games append to `games.json`. You can view your game history.
CP06-ai	Groq Wired Up	API key in `.env`, first Groq call works, PvP/PvAI toggle works. AI plays legal moves.
CP07-levels	Difficulty and Personality	Three difficulties, three personalities. AI returns `{move, comment}` as structured JSON.
CP08-stats	Leaderboard and AI Stats	Leaderboard ranks players. AI stats page shows win rate by difficulty and personality.
CP09-c1	Custom Feature #1	Your idea. Spec it in a mini-PRD before you build it.
CP10-c2	Custom Feature #2	Your second idea. Same rule.

The Checkpoints Page (Required)

This is the most important page in your app. It's public (no login required) and it's your portfolio.

For every checkpoint you complete, your Checkpoints page must show:

1. **The tag** (e.g., CP04-pvp)
2. **A link to the GitHub commit or PR** for that checkpoint
3. **What I built** (2-3 sentences, plain English)
4. **What was hard** (1-2 sentences, honest)
5. **What I'd do differently** (1 sentence)
6. **AI usage note** (1 sentence: where AI helped, where you overrode it)
7. **A/B testing note** (1 sentence: what approaches or prompts you tried, which one you kept, and why)

Optional but encouraged: a screenshot or a short Loom video.

You update this page every work day. It is not something you write at the end.

Grading

You are graded on your process, not just your app. An app that works but has empty Checkpoint reflections is worth less than an app with one bug and thoughtful reflections on every checkpoint.

How Points Break Down

Area	**Weight**	**What I'm Looking For**
Checkpoints tagged correctly in GitHub	30%	All 10 checkins exist in github. Commits are clean.
Checkpoints page (the reflections)	30%	Honest, specific, thoughtful. Vague entries lose points.
Custom features (CP09 and CP10)	30%	Originality, difficulty appropriate for your skill, working end to end. Each has its own mini-PRD.
App functionality	10%	The required features work.

Notice: the app itself is only 10%. If that surprises you, reread the top of this document.

What Gets You Full Credit on Reflections

Good: "I tried asking Groq for just a move number, then switched to asking for JSON with `{move, comment}`. The JSON version was easier to parse but the AI sometimes returned invalid JSON. I added a try/catch and a fallback random move."

Not good: "It was hard but I got it working with AI's help."

The difference: specifics, decisions, and evidence you actually thought about it.

What Gets You Zero Credit

- Committing your `.env` file or API key
- Copying another student's code and tagging it as your own
- Writing reflections that could apply to anyone (they must be about what *you* actually did)
- Missing checkpoint tags or broken GitHub links on your Checkpoints page

Definition of Done

Your project is complete when:

1. All 10 checkpoints are tagged in GitHub.
2. Your Checkpoints page is live, complete, and linked from the home page.
3. A classmate can clone your repo, follow your README, and run the app in under 5 minutes.
4. You can demo every feature live in under 3 minutes.
5. Your README has a screenshot, setup steps, and a one-paragraph reflection on what you learned.

How to Ask for Help

1. **Try for 15 minutes.** Read the error. Read your code. Read the error again.
2. **Ask AI for help.** Paste the error, ask what's wrong, ask *why* before you accept the fix.
3. **Ask a classmate.** Peer debugging is faster than you think.
4. **Ask me.** Bring: what you tried, what you expected, what actually happened.

If you skip straight to step 4, I'm going to walk you back to step 1.

One Last Thing

The work you do in this project becomes a public GitHub repo with your name on it. A year from now, when you apply for an internship, a program, or a first job, this is the kind of thing you show people. Build something you'll be proud to link to.